江西省智慧应急综合
业务平台
开发标准规范

江西省应急管理厅　　发 布

目　录

1. 范围 7

2. 规范性引用文件 7

3. 术语和定义 7

3.1. 卡片 7

3.2. 模块 7

3.3. 4A 8

3.1 VSCODE 8

3.2 GIT 8

3.3 微前端 8

3.4 卡片门户 8

3.6 代码生成器 8

4. 建设模式 9

5. 基础要求 9

5.1. 环境要求 9

5.2. 技术栈要求 9

5.2.1. 前端技术栈 9

5.2.2. 后端技术栈 10

5.3. 基础支撑要求 10

5.3.1. 4A 10

5.4. 准出要求 10

5.4.1. 性能要求 10

5.4.2. 样式要求 11

5.4.3. 代码质量要求 11

5.4.4. 代码交付要求 11

6. 开发规范 11

6.1. 前端开发规范 11

6.1.1. 接入流程说明 11

6.1.2. 卡片开发 12

6.1.2.1. 创建卡片 12

6.1.2.2. 目录结构 14

6.1.2.3. 发布卡片 17

6.1.3. 模块开发 18
6.1.3.1. 创建模块 18
6.1.3.2. 目录结构 19
6.1.3.3. 构建模块 22
6.2. 后端开发规范 23
6.2.1. 编码规范 23
6.2.2. 技术组件 23
7. 附录 A（JXSTJH_CHECKSTYLE.XML） 24
8. 附录 B（JXSTJH_CODE_SCHEME_IDEA.XML） 24
9. 附录 D（版本服务器申请） 25

前　言

本规范按照 GB/T1.1-2020 《标准化工作导则 第一部分：标准化文件的结构和起草规则》的规定编写。

本规范主要起草单位：江西省水投江河信息技术有限公司。

引　言

为进一步明确省智慧应急项目建设工作，科学、规范地指导各单位系统开发与对接，特制定此规范。

本规范是规定了江西省智慧应急综合业务平台开发规范。

本规范用以江西省智慧应急项目开发规范推进建设。

江西省智慧应急综合
业务平台
开发标准规范

1. 范围

本规范规定了江西省智慧应急项目对接综合业务平台的建设模式、基础要求、开发规范。

本规范适用于指导各单位智慧应急系统项目入综合业务平台建设工作的规划、制定、修订、应用与管理等工作。

2. 规范性引用文件

下列标准对于本规范的应用是必不可少的。凡是注日期的引用文件，仅注明日期的版本适用于本文件。凡是不注明日期的引用文件，其最新版本适用于本文件。

GB/T 32619-2016 政务服务中心信息公开编码规范

3. 术语和定义

下列术语和定义适用于本文件。

3.1. 卡片

卡片是业务平台的一种展示形式，支持注册发布、自由拖动布局，样式上类似卡片模式的UI展示形式。是业务平台的容器实体，承载着各业务应用的业务逻辑封装，本质上是一个遵循了现代前端开发umd规范的JavaScript模块文件。umd模块是一个兼容amd标准、commonJs标准及es6标准的通用模块，有着良好的兼容性，卡片文件中将业务代码，隔离的样式作为JavaScript模块资产进行封装。

3.2. 模块

模块是业务平台的另一种展示形式，支持业务系统各功能重组展示。

3.3. 4A

统一身份认证平台(4A)是提供支持密码、二维码、移动端生物认证（指纹、面容等）等认证方式，满足对内外部应用提供多种认证协议的统一认证服务的支撑平台。

3.1 VSCode

VSCode（Visual Studio Code）是一款开源免费，支持自定义配置、智能提示强大、支持各种文件格式（html/jade/css/less/sass/xml）、调试功能强大、各种方便的快捷键、集成 git、强大的插件扩展轻量级的好用的编译软件。

3.2 GIT

Git 是用于 Linux 内核开发的版本控制工具。与常用的版本控制工具 CVS, Subversion 等不同，它采用了分布式版本库的方式，分布式相比于集中式的最大区别在于开发者可以提交到本地，每个开发者通过克隆（git clone），在本地机器上拷贝一个完整的 Git 仓库，不必服务器端软件支持，使源代码的发布和交流极其方便。

3.3 微前端

微前端（micro-frontend）是近几年在前端领域出现的一个新概念，主要内容是将前端应用分解成更小、更简单的能够独立开发、测试、部署的小块。微前端的理念源于微服务，将庞大的整体拆成可控的小块，并明确他们之间的依赖关系，而它的价值在于能够将低耦合的代码组件进行组合。

3.4 卡片门户

承载各式卡片的容器页面，网格化的设计使得卡片在工作台中可以自由拖拽大小及位置排序，依据实际业务需求拖入满足业务场景的卡片，使得门户工作台呈现出千人前面的个性化配置。

3.6 代码生成器

一款开发编辑器拓展插件，提供快速生成卡片工程能力，通过插件生成的工程项目中包含开发，预览，发布构建的快捷指令。降低卡片开发前期成本，规范工程结构及提供部分代码使用示例。

4. 建设模式



业务平台采用微前端升级的业务门户架构可以将一个庞大的应用群从烟囱堆叠式改成中心化，架构的优越性应用使构建速度更快、更新迭代更容易、依赖管理更清晰，业务应用更加专注自身业务实现而非架构等其他因素。

业务平台作为开放式能力平台，提供卡片化、模块化、外部页面配置三种模式，平台推荐使用卡片及模块化接入模式。

5. 基础要求

5.1. 环境要求

要求整体代码及各类中间件、组件适配 centos7、chrome69 及以上版本、firefox52 及以上版本。

5.2. 技术栈要求

5.2.1. 前端技术栈

前端请使用 VUE2.X 版本，开发环境正确安装和配置了 Node.js V8.9 或以上及 vscode 开

发工具，技术栈清单：

技术项名称	强制	建议	可选	版本	技术栈说明
vue	☼			v2.2↑	开发框架
@jxstjh/ant-design-vue	☼			v1.6.5↑	基础 ui 组件库
@jxstjh/portal	☼			v0.0.27↑	门户公共业务组件库
@jxstjh/vue-module-loader	☼			v0.0.10↑	js 模块加载器
vue-cli		☼		v3.0↑	Vue 开发脚手架
vuex		☼		v3.0↑	Vue 应用状态管理器
vue-router	☼			v3.1↑	Vue 官方路由
ECMAScript		☼		Es2015↑	js 编程语言版本
Less			☼	v3.0↑	css 预处理扩展
axios		☼		v0.19↑	网络请求库
jh-project-generator		☼		最新版本	模块代码生成工具
arcgis		☼		v4.0↑	ArcGISAPIforJavaScript
vue2-perfect-scrollbar			☼	v1.2.0↑	滚动条美化插件

5.2.2. 后端技术栈

我们强烈建议使用 java spring boot 或者 spring cloud 进行后台逻辑开发。

5.3. 基础支撑要求

5.3.1. 4A

采用统一的用户及认证体系，业务平台仅支持授权模式为“普通模式”，认证协议为“jwt”的外部系统资源，4A 相关接入资料及代码请参考附录 C（4A 接入资料）。

5.4. 准出要求

5.4.1. 性能要求

平台运行在信创环境下，结合信创环境硬件性能，单卡片或模块页面 2S 内完成渲染的标准。

5.4.2. 样式要求

遵循《江西省智慧应急综合业务平台界面设计与交互规范》。

5.4.3. 代码质量要求

前后端代码规范及各中间件使用规范遵循《江西省智慧应急统一开发管理规范》

后端 checkStyle 配置请参考附录 A（jxstjh_checkstyle.xml）

后端代码自适用配置请参考附录 B（jxstjh_code_scheme_idea.xml）

5.4.4. 代码交付要求

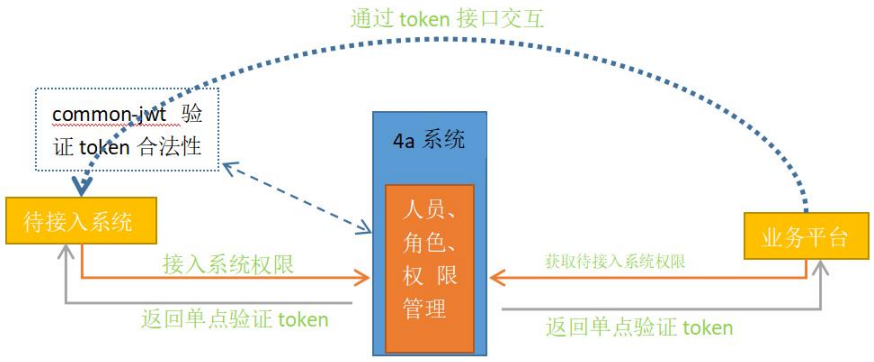
完整源代码上传至江河 GIT 版本服务器，环境申请流程参考附录 D（版本服务器申请）。

6. 开发规范

6.1. 前端开发规范

遵守卡片及模块相关使用说明及规范，公共组件及更多详情请查看在线文档 https://guining.gitee.io/jxstjh-portal/#/docs/vue/getting-started-cn/。

6.1.1. 接入流程说明



卡片及模块接入前提为系统已接入 4A 系统的单点登录：

卡片接入：待接入系统开发人员按业务平台前端组件样式规范编写卡片前端代码，卡片内部数据由待接入系统后台接口提供，待接入系统的前后端交互需要使用业务平台提供的 token。

模块接入：业务平台需要去 4A 获取当前人员拥有 A,B,C 中哪些菜单的权限，根据 4A 权限配置获取人员菜单权限，待接入系统模块中人员身份判断则需要解析 token 获取身份信息。

6.1.2. 卡片开发

6.1.2.1. 创建卡片

1. 安装 vscode 插件

   Visual Studio 插件 jh-project-generator，

   下载地址 https://marketplace.visualstudio.com/items?itemName=jh-team.jh-project-generator

2. 创建一个卡片工程

   我们通过插件向 vscode 注册了一个江河前端代码生成器命令了解 vscode 命令.

   打开江河前端代码生成器

   可以使用命令面板调用。

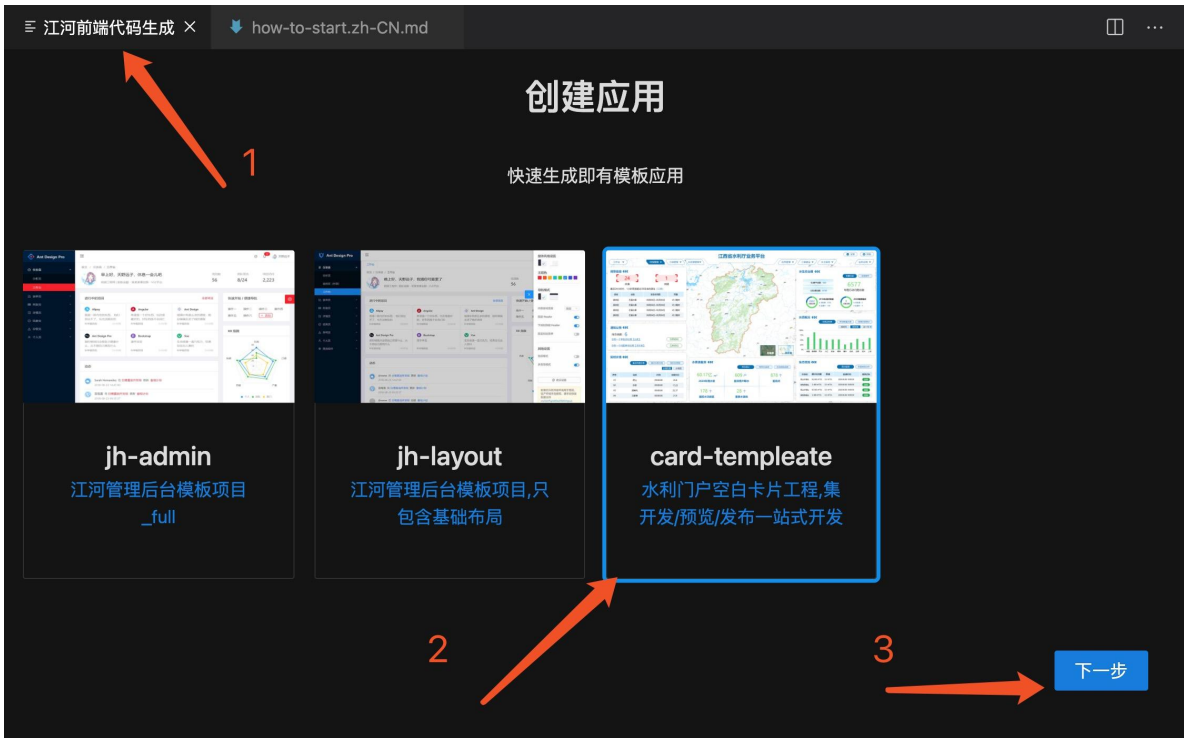
   可以使用键绑定来调用。

   可以通过 VS Code UI 调用，例如通过编辑器标题栏。

   例:默认情况下使用 ctrl+Shift+p(command+Shift+p)或在命令行中输入江河前端代码生成器

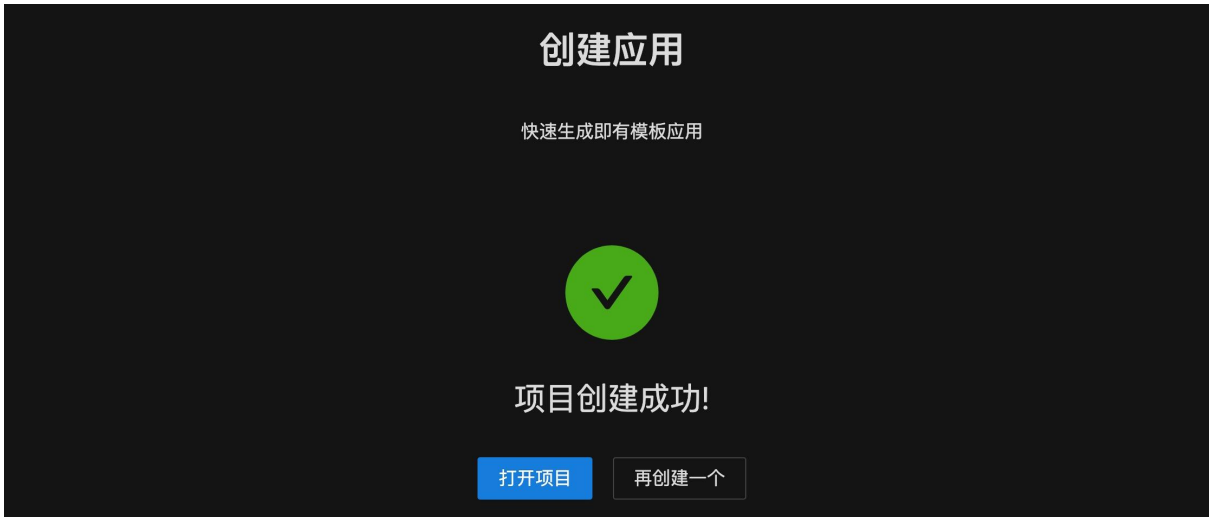


   vscode 将自动打开一个 webview 标签页，接下来选择对应的卡片模板后选择**下一步**.



输入项目名称及选择项目生成位置点击**完成**

注:您可能会碰见网络问题,如遇见问题请联系相关人员获取权限!





点击打开项目按钮则创建卡片成功!

6.1.2.2. 目录结构

1. 脚手架目录

```
├── ...
├── dist                          # 产出文件夹
│   ├── img                        # 卡片需要用的静态资源等
│   ├── jh-test-card.umd.min.js    # 卡片生产 js
├── example                       # 卡片预览工程
├── src                           # 卡片工程
│   ├── assets                     # 需要使用的静态资源
│   ├── components                 # 组件文件夹
```

```
|   |   ├── JhTestCard.vue          #@ 基于项目名称自动生成的组件,提供给门户的唯一组件
|   |   └── Logo.vue                # 相关业务组件
|   ├── module.js                   #@ 卡片模块根文件
|   ├── router                      # 如果您需要拓展门户页面
|   └── store                       # 如果您需要拓展门户 vuex
├── vue.config.js                   # 标准的 vue-cli 文件
├── package.json                    # 标准的 package.json 文件
└── ...
```

注：带有#@前缀的文件是插件根据您的项目名称自动生成的,如果您有修改的需要,请您在明确知道修改项后再操作

2. 目录及说明

1) module.js

卡片打包入口文件,此文件是卡片接入门户的入口文件,卡片的混入门户逻辑在此处编写,您编写的卡片将获得以下能力【注册卡片(组件)到门户、事件通信】

```
import JhTestCard from './components/JhTestCard.vue'
import routes from './router/routes'
import storeModule from './store/store-module'
export default function (Vue) {
  Vue.config.productionTip = false
  // 向门户添加业务页面
  this.$router.addRoutes(routes)
  // 向门户添加Vuex store实例
  this.$store.registerModule('main', storeModule)
  // 监听门户自定义事件
  this.$eventBus.on('visitedAbout', () => {
    alert('用户访问了about页面。')
  })
  /* 向门户添加卡片组件
  ** 目前一个工程只支持注册一个组件
  ** 但是你可以在入口组件中引入其它业务组件.
  */
  this.$dynamicComponent.create(JhTestCard)
}
```

2) JhTestCard.vue

本质上就是一个普通的 vue 组件, 再本工程里额外会作为动态组件在门户运行时加载进门户项目. 除此之外和正常组件没有区别

```vue
<template>
  <div >
    <JPortalcard :title="'测试卡片777'" :bordered="false">
      <a-button type="primary" @click="openDialog">
        打开弹窗
      </a-button>
      <span class="hello">card 07 </span>
    </JPortalcard>
  </div>
</template>

<script>
import Logo from './Logo.vue'
export default {
  // package.json中的name
  name: 'JhTestCard',
  components: {
    // eslint-disable-next-line vue/no-unused-components
    Logo
  },
  props: {
    msg: {
      type: String,
      default: null
    }
  },
  created () {
    // 示例代码(展示使用基座项目api打开弹出层显示吐司消息) 开发时请自行移除
    this.$message.info('hello word 我来自远程.共享基座项目vue实例!')
  },
  methods: {
    // 示例代码(展示使用基座项目api打开弹出层对话框) 开发时请自行移除
    openDialog () {
      this.$dialog(Logo,
        // component props
        {
          record: {},
          on: {
            ok () {
              console.log('ok 回调')
            },
            cancel () {
              console.log('cancel 回调')
            },
            close () {
              console.log('modal close 回调')
            }
          }
        },
        // modal props
        {
          title: '新增',
          width: 700,
          centered: true,
          maskClosable: false
        })
    }
  }
}
</script>
<style  scoped>
  .hello{
    color:red
  }
</style>
```

3) package.json

package. json 中有几个关键字段需要额外说明



```
{
  "name": "jh-test-card",
  "version": "0.0.1",
  ...
  "scripts": {
    "serve": "cd example && npm run serve",
    "build": "vue-cli-service build --target lib src/module.js"
  },
  "dependencies": {
    ...
  },
  "devDependencies": {
    ...
  }
}
```

a. name 卡片的工程名称,也就是创建卡片时输入的名称. 必须和发布卡片时候的组件名一致，否则导致发布失败

b. version 卡片的工程的版本号

c. card_config

- width 卡片的初始宽度
- height 卡片的初始高度
- gis 卡片是否需要 gis 联动

d. scripts

- npm run serve 将运行预览工程,开发卡片时使用
- npm run build 构建标准门户卡片
- npm run pub 构建标准卡片&&生成工程压缩包以供上传 v0. 1. 3

6.1.2.3. 发布卡片

1) 构建卡片

开发人员在卡片工程中开发完成,并确保 package. json 中需要配置的配置项填写正确后,在工程中运行

npm run pub

开发人员将会在工程根目录得到一个根据 package. json 中的 name 字段构建好的 zip 包。

2) 卡片发布

按《江西省智慧应急综合业务平台上线发布规范》要求完成卡片发布。

6.1.3. 模块开发

6.1.3.1. 创建模块

1. 安装 vscode 插件

   Visual Studio 插件 jh-project-generator，

   下载地址

https://marketplace.visualstudio.com/items?itemName=jh-team.jh-project-generator

2. 创建一个模块工程

   我们通过插件向 vscode 注册了一个江河前端代码生成器命令了解 vscode 命令.

   打开江河前端代码生成器

   可以使用命令面板调用。

   可以使用键绑定来调用。

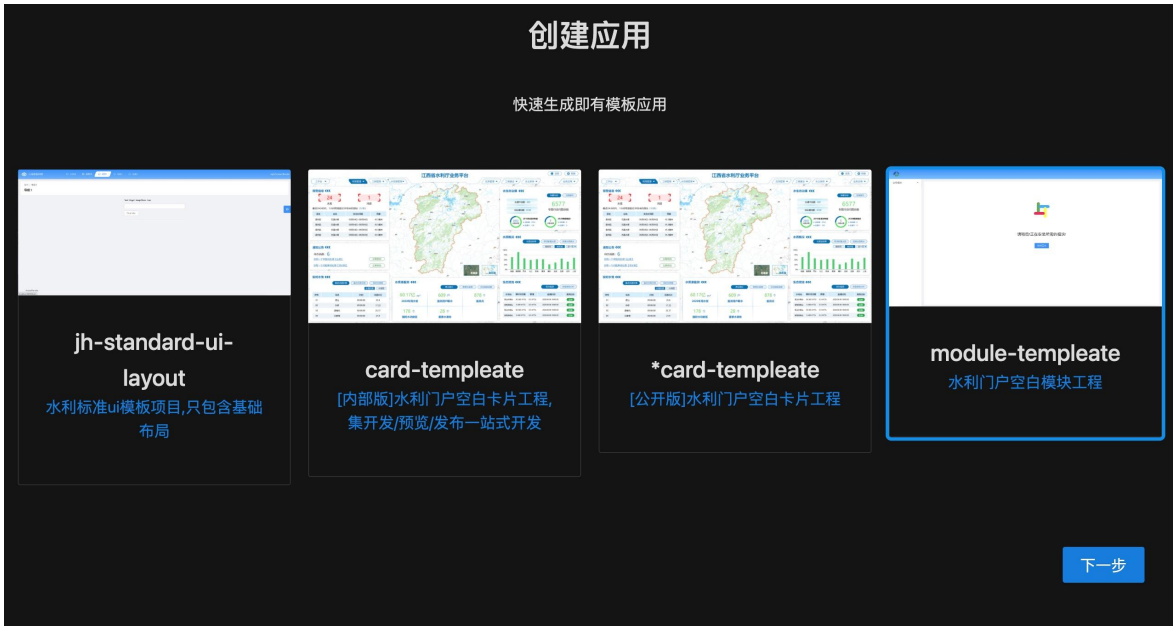
   可以通过 VS Code UI 调用，例如通过编辑器标题栏。

   例:

   默认情况下使用 ctrl + Shift + p (command + Shift + p)

   在命令行中输入江河前端代码生成器

   vscode 将自动打开一个 webview 标签页,接下来选择对应的模块模板后选择下一步.

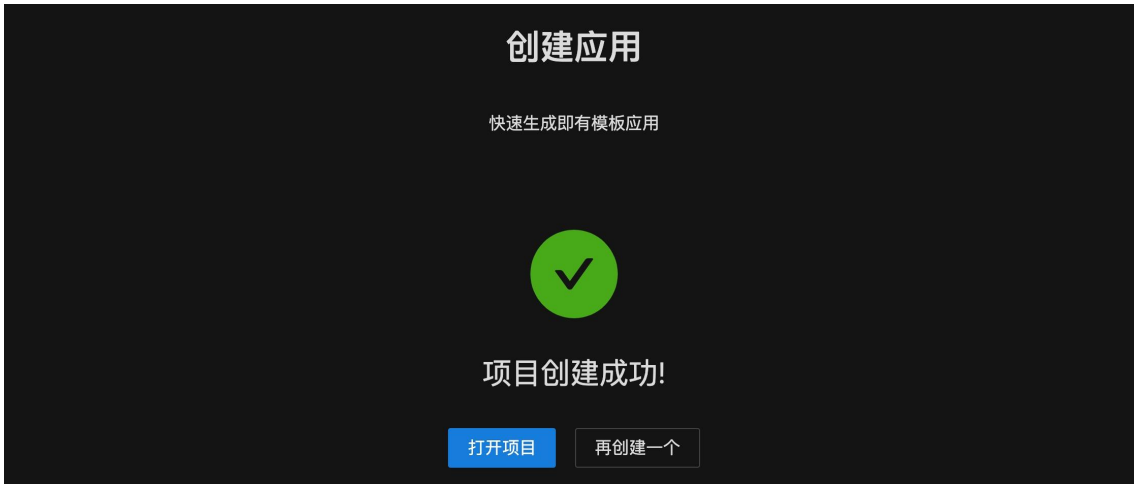


输入项目名称及选择项目生成位置，输入资源 code 注:统一安全管理平台中应用的 resourceCode

点击完成。

注:您可能会碰见网络问题,如遇见问题请联系相关人员获取权限!





点击打开项目按钮则创建模块成功!

6.1.3.2. 目录结构

1. 脚手架目录

```
├── ...
├── dist                              # 产出文件夹
│   ├── img                           # 模块需要用的静态资源等
│   ├── jh-test-module.umd.min.js     # 模块生产 js
├── example                           # 模块预览工程
```

```
├──── src                          # 模块工程
|    ├──── assets                   # 需要使用的静态资源
|    ├──── components               # 组件文件夹
|    ├──── module.js                # 模块根文件
|    ├──── router
|    |    └──── index.js             #@ 拓展门户路由配置表
|    └──── store                    # 拓展门户 vuex
|    └──── views                    # 页面组件
├──── vue.config.js                # 标准的 vue-cli 文件
├──── package.json                 # 标准的 package.json 文件
└──── ...
```

注：带有#@前缀的文件是插件根据您的项目名称自动生成的，如果您有修改的需要，请您在明确知道修改项后再操作

2. 目录及说明

1) module.js

模块打包入口文件，此文件是模块接入门户的入口文件，模块的混入门户逻辑在此处编写，您编写的模块包将获得以下能力【操作门户路由、向门户 Vuex 中添加 store 实例、事件通信】

```
import routes from './router'
import storeModule from './store/store-module'
const { name } = require('../package.json')
export default function (Vue) {
  Vue.config.productionTip = false
  // 挂载路由配置表
  this.$mountRoutes(name, routes)
  // 挂载有命名空间的store实例
  this.$store.registerModule(name, storeModule)
}
```

2) router/index.js

vue-router 的路由配置对象，在本工程里对路由对象有一些额外的要求：

a. 模块的路由配置都书写在更节点 children 内
b. 模块内的菜单路由应在统一安全管理平台中注册
c. 不支持 webpack 惰性加载特性
d. 深层的嵌套路由使用 routerView 包裹

```
import AboutMe from '../views/AboutMe.vue'
import { name } from '../../package.json'
const routeView = {
  name: `${name}-root`,
  render: (h) => h('router-view')
}
export default [
  /** not allow script **/
  {
    path: name, // 固定格式 勿修改
    url: name, // 固定格式 勿修改
    component: routeView, // 固定格式 勿修改
    children: [
      /** allow script **/
      // children 内容为当前模块包含的所有页面
      {
        path: 'about-me1',
        name: 'about-me1',
        meta: { a: { b: 1 } },
        component: AboutMe
      }
      /** allow script **/
    ]
  }
  /** not allow script **/
]
```

3) package.json

package. json 中有几个关键字段需要额外说明



```
{
  "name": "jh-test-module",
  "version": "0.0.1",
  "scripts": {
    "zip": "node script/zip.js",
    "pub": "npm run build && npm run zip",
    "generateRouter": "node ./script/generateRouter.js",
    "predev": "npm run generateRouter",
    "prebuild": "npm run generateRouter",
    "dev": "vue-cli-service build --watch  --target lib src/module.js --dest ./example/public/dist",
    "build": "vue-cli-service build --target lib src/module.js"
  },
  "dependencies": {
    ...
  },
  "devDependencies": {
    ...
  },
  "module_config": {
    "resourceCode": "991",
    "pages": [
      {
        "name": "about-me1",
        "path": "about-me1",
        "redirect": null,
        "meta": {
          "a": {
            "b": 1
          }
        },
        "children": null
      }
    ]
  }
}
```

a. name 模块的工程名称,也就是创建模块时输入的名称.发布/更新模块时会业务平台会自动识别

b. version 卡片的工程的版本号

c. module_config

- resourceCode 统一安全管理平台分配的 resourceCode
- pages 默认会由 router/index.js 自动生成的配置,特殊情况可以手动配置

d. scripts

- npm run dev 开发模块时使用
- npm run serve:example 将运行预览工程,开发模块时使用
- npm run build 构建标准模块产物
- npm run pub 构建标准模块产物&&生成工程压缩包以供上传 v0.1.3

6.1.3.3. 构建模块

1) 构建卡片

开发人员在模块工程中开发完成,并确保 package.json 中需要配置的配置项填写正确后,在工程中运行

```
npm run pub
```

开发人员将会在工程根目录得到一个根据 package.json 中的 name 字段构建好的 zip 包。

2) 模块发布

按《江西省智慧应急综合业务平台上线发布规范》要求完成模块发布。

6.2. 后端开发规范

6.2.1. 编码规范

遵循《江西省智慧应急统一开发管理规范》。

6.2.2. 技术组件

提供包括日志组件、监控组件、redis 缓存等功能组件，具体参见在线文档：http://generator.dev.jxwrd.gov.cn/#/docs

7. 附录 A（jxstjh_checkstyle.xml）



jxstjh_checkstyle.xml

8. 附录 B（jxstjh_code_scheme_idea.xml）



jxstjh_code_scheme_idea.xml

9. 附录 D（版本服务器申请）

Gitlab 账号开通申请表

申请人		人员类型	☐内部 ☐外部
申请人邮箱		联系方式	
申请周期		组别	☐前端组 ☐后端组
项目名称		角色权限	☐ Guest ☐ Reporter ☐ Developer ☐ Maintainer
申请原因			
项目联系人		联系方式	

年 月 日

Gitlab 项目开通申请表

申请人		联系方式	
项目名称			
项目描述			
项目后端组名称（英文）	*eg. jh-dingtalk*		
项目前端组名称（英文）	*eg. jh-dingtalk-ui*		
分支（自填）	*eg. master、developer、test*		
申请原因			

年　　月　　日